



DEAKIN UNIVERSITY

SIT314 Software Architecture and Scalability for Internet-of-Things

TRIMESTER 2, 2026

FACULTY OF SCIENCE, ENGINEERING AND BUILT ENVIRONMENT
SCHOOL OF ENGINEERING

**BMOAS Further Revision Report
Architecture and Measured Scalability**

Peter William Davidoff

Improving the availability of audience ratings

BMOAS is a simulated service for measuring exposure to media. Meters identify a media source and send observations that contribute to ratings by source, region and reporting period. The service must retain distinct observations, recognise repeat deliveries and count each participant once within a ratings bucket. Processing delay affects how soon these summaries are available after an event.

This report follows successive changes made in response to the tutor's feedback about slow autoscaling and bottlenecks in Node-RED and validation. The first revision improved performance but missed the original timing targets. Further work reduced aggregation overhead, prepared capacity before expected demand and divided shared ratings counters into smaller storage groups. The original observations and ratings rules were retained throughout.

Successive changes

Stage	Change and reason
First revision	Reduced repeated validation queries, batched SQS requests and enabled scaling across all processing stages.
Batched aggregation	Processed up to ten observations in a transaction. Prepared four tasks per stage before demand.
Partitioned ratings	Spread each logical ratings bucket over 64 groups to reduce concurrent writes to the same record. Combined the groups when reporting.

The cloud path remains simulated Node.js meters, MQTT over TLS, AWS IoT Core, SQS, Node-RED, validation, MongoDB Atlas, aggregation and an authenticated reporting API. ECS services and Atlas are hosted in AWS Sydney. The redesign uses conventional batching, partitioning and capacity management within the Distinction project.

Ratings remain consistent

A stable hash of the participant identifier selects a storage group. Membership identifiers still use the logical reporting bucket and participant identifier, so repeat observations cannot increase the participant count twice. MongoDB transactions commit membership changes, counters and applied observation markers together. Confirmation is recorded after commit. The API combines the storage groups into the original response format. This partitions application counters within the existing Atlas replica set.

Capacity management and evaluation method

Expected news and sporting events can justify preparing capacity before demand arrives. Historical ratings and the programme schedule can inform this forecast, together with the number of active meters and their reporting frequency. Listenership alone does not change the arrival rate when a fixed fleet reports continuously at a fixed interval.

The configurable example timetable raises the minimum to four tasks at 05:45 and 17:45 in Australia/Sydney time. It lowers the minimum to two at 10:00 and one at 23:00. These times are illustrative. The maximum remains four, allowing reactive scaling during quieter periods. Lowering the minimum lets the existing policies reduce capacity as demand falls rather than forcing an immediate shutdown.

A separate scheduled action was tested in AWS. All three processing stages increased from one to four running tasks within 63.84 seconds of the scheduled time. Load began after that capacity was ready. Recurring schedules were disabled during the controlled trials. This separates the preparation mechanism from the earlier reactive scaling experiments.

Final workload	Configured demand	Original target
Baseline	100/s for 600 s, 60,000 observations	p95 below 2 s
Higher load	1,000/s for 120 s, 120,000 observations	p95 below 5 s
Reconnect	50,000 buffered plus 12,000 regular observations	Recovery within 120 s

The earlier batched version also ran the original 900 second higher-load profile with 900,000 observations. The partitioned version used 120 seconds within the remaining evaluation allowance. Each final workload has one trial. The shorter interval and changes to capacity prevent a controlled attribution of improvement to one change alone.

Whole-path latency starts at the first MQTT publication attempt and ends at confirmed aggregation commit. p95 covers unique completed observations. Throughput divides completions by the time from first publication to final confirmation, including recovery. Recovery starts when publication finishes. Offline buffering time is excluded. Exact identifier sets and expected ratings are reconciled for every trial.

Four tasks run at each processing stage. Each Node-RED and validation task has one vCPU and 2 GiB, while each aggregation task has half a vCPU and 1 GiB. CloudWatch records CPU, memory and queues. The deployment retains private ECS networking, scoped IAM roles, secret references, TLS and restricted Atlas access.

Results against the timing targets

Attempt and workload	Observations	p95	Outcome
First revision baseline	60,000	92.358 s	Above 2 s
Batched baseline	60,000	0.228 s	Met 2 s target
Batched higher load 900 s	900,000	242.231 s	Above 5 s
Partitioned baseline	60,000	0.226 s	Met target
Partitioned higher load 120 s	120,000	0.288 s	Met target
Partitioned reconnect	62,000	0.506 s	Recovery 0.149 s

Table 1. The first revision baseline began with one task per stage. Later trials prepared four. The higher-load trials have different durations. Reconnect is assessed against its recovery target rather than the baseline p95 target.

The first revision improved the matched stepped trials: mean p95 fell from 587.45 to 367.34 seconds, throughput rose from 178.46 to 229.60 observations/s and recovery fell from 528.75 to 304.97 seconds. These improvements retained exact ratings but still left substantial delays. The next batched version met the baseline target. At higher load, aggregation fell behind despite prepared capacity, indicating a further constraint beyond task startup.

The final reconnect trial recovered in 0.149 seconds after publication ended. The next empty-queue sample was at 28.32 seconds. It completed 62,000 observations. Its identifier and ratings checks are recorded in Appendix B.

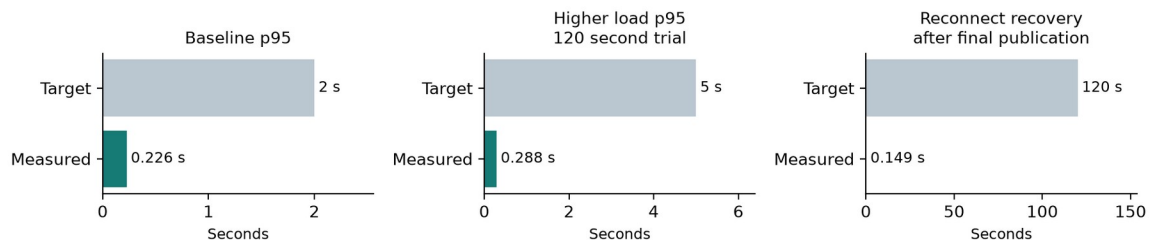


Figure 1. Final measured latency and recovery compared with the original targets. Durations and task allocations are stated above.

Interpretation and application outcome

Preparing capacity removes the wait for an alarm and task startup from the beginning of an expected peak. The scheduled demonstration supports using programme and ratings schedules for BMOAS. An operational lead time should include the measured startup time and a suitable margin. Unexpected demand still requires monitoring and reactive scaling.

Partitioning reduces simultaneous writes to shared counters. The reporting query then combines their counts. This trades additional stored counters and query work for less contention during ingestion. The final tests preserve the media ratings calculation and the treatment of duplicate and late observations. Separate evaluation of concurrent reporting demand would help assess the query cost.

Verification passed 58 unit tests and two real MongoDB integration tests. Tests covered overlapping batches, duplicate observations, rollback, missing records, late observations, no-media observations and confirmation repair. The reported ratings matched the original aggregation logic. Earlier cloud checks also verified rejection of invalid messages and HTTP 401 without an API token and HTTP 200 with one.

The generator's original 20 millisecond spacing across 20 gateways set a theoretical ceiling of 1,000 publications/s before execution overhead. The final version uses 12 milliseconds to provide headroom while keeping the configured generation rate and event contract unchanged. The measured publication and completion rates accompany the results.

Scope of the findings

These trials use synthetic observations, one region and one Atlas tier. The final higher-load trial lasts two minutes and each final configuration has one repetition. The earlier 900,000 observation run was allowed to drain with unchanged software and capacity after its collector ended. Its full identifiers, latency and ratings were recovered, while continuous queue and task samples cover only part of recovery. The results describe these measured conditions.

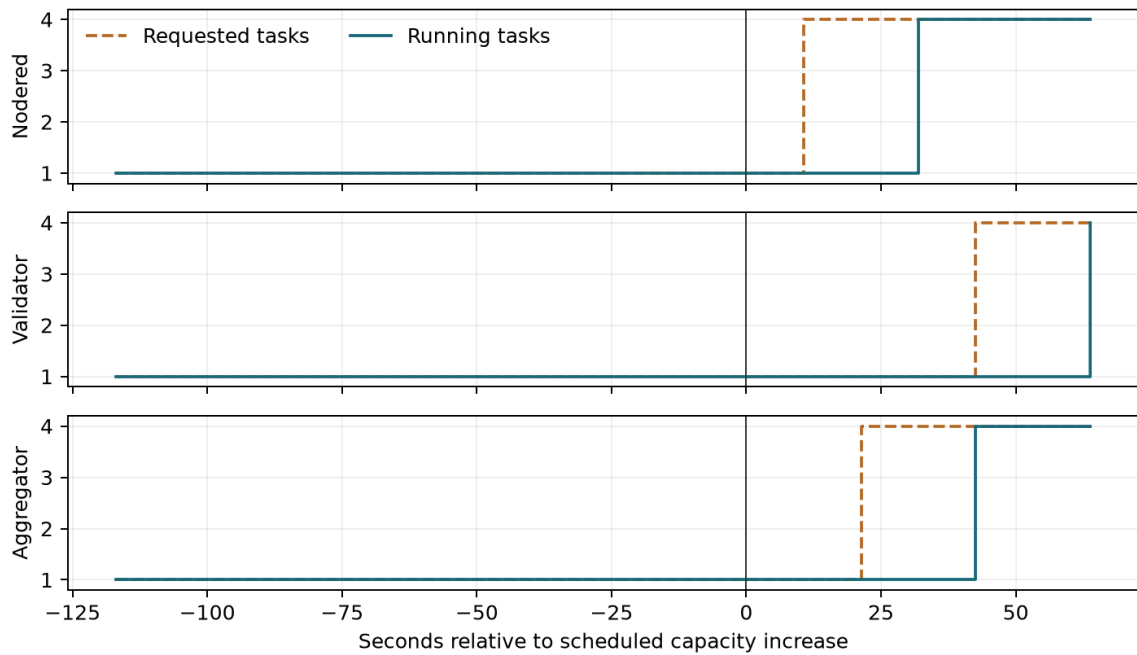
Under the final measured conditions, BMOAS met the baseline latency, higher-load latency and reconnect recovery targets while preserving exact observations and ratings. The successive tests show why improving the complete processing path and preparing capacity matters to timely audience measurement.

AI acknowledgement

ChatGPT and Codex generated implementation code and assisted with configuration, tests, deployment, troubleshooting, analysis, figures and this report. I reviewed the work and remain responsible for the submission, the interpretation of results and explaining the architecture.

Appendix A Deployment and capacity evidence

Capacity prepared before the workload



All three stages reached four running tasks after 63.84 seconds. Samples are approximately 10 seconds apart.

Figure A1. Recorded requested and running task counts during the scheduled increase. All stages were ready before the baseline workload began.

Revised stepped trial — running tasks by processing stage

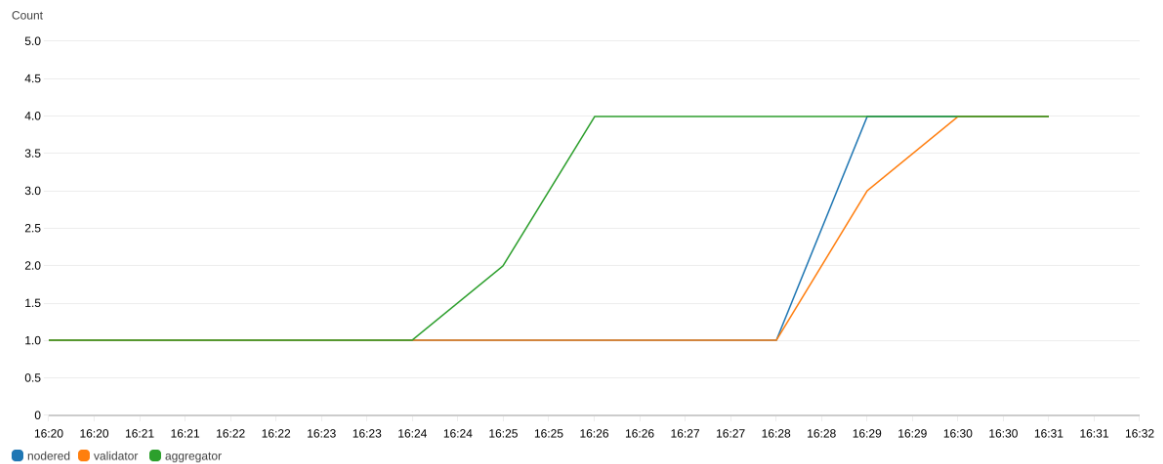


Figure A2. CloudWatch evidence from the first revision, showing capacity arriving during the stepped workload.

Appendix B Reconciled experiment evidence

Trial	Completed	p95 s	Recovery s
original-fixed-r1	60,000	245.993	250.006
original-reconnect-r1	62,000	521.172	490.620
original-stepped-r1	180,000	576.484	516.448
original-stepped-r2	180,000	598.407	541.060
revised-fixed-r1	60,000	92.358	86.634
revised-reconnect-r1	62,000	305.357	275.980
revised-stepped-r1	180,000	341.893	277.134
revised-stepped-r2	180,000	392.789	332.800
target-baseline-r1	60,000	0.228	0.232
target-scale-r1	900,000	242.231	242.618
partitioned-baseline-r1	60,000	0.226	0.123
partitioned-reconnect-r1	62,000	0.506	0.149
partitioned-scale-r1	120,000	0.288	0.215

These trials reconcile generated identifiers with completed observations and compare ratings with counts independently derived from stored observations. The batched higher-load trial includes an unchanged extended drain. Its original collection record is retained.

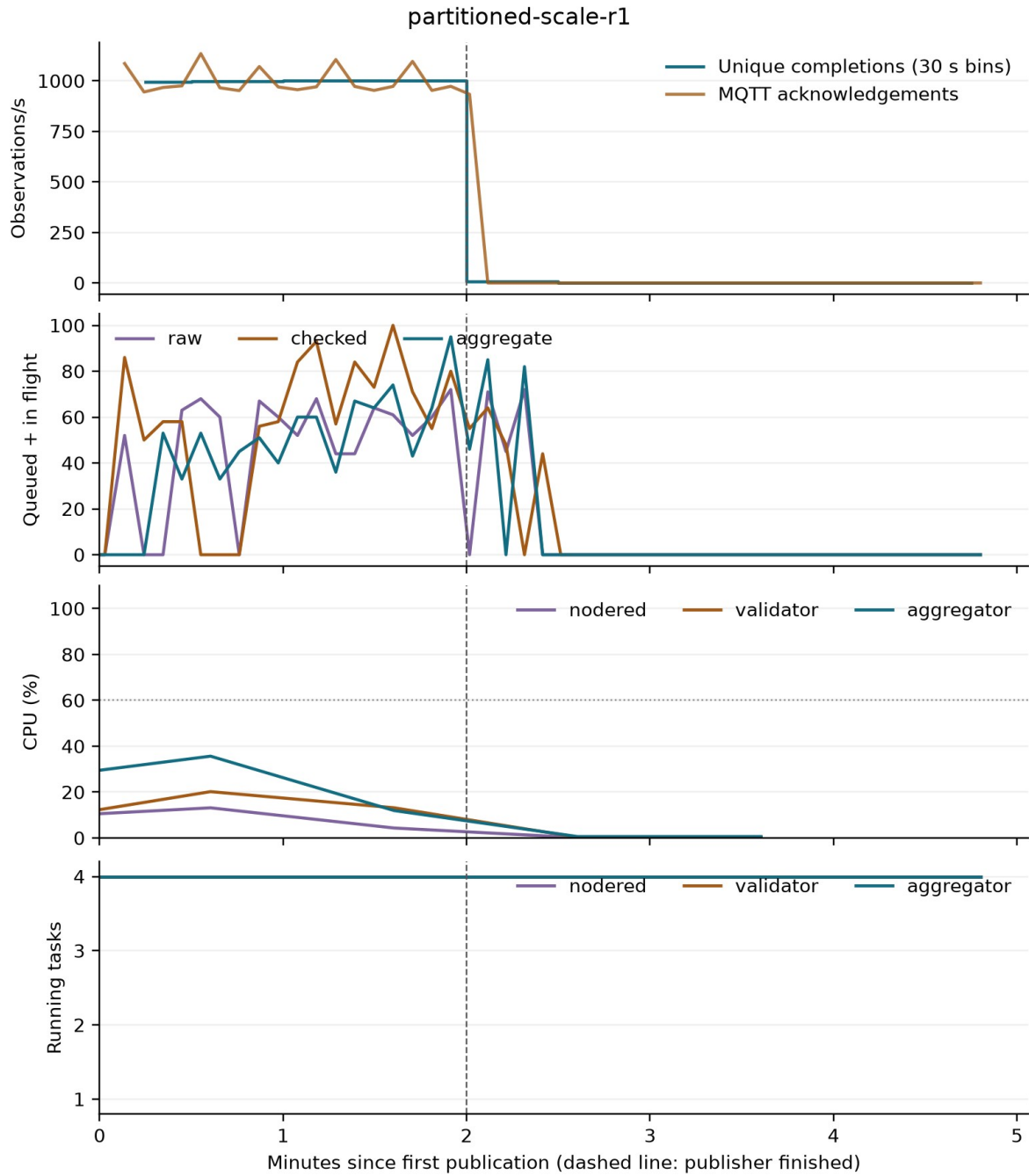


Figure B1. Final higher-load processing, queues and service measurements.

The original report and first revision report are preserved. The private repository contains source, configuration and test material at <https://github.com/pdavidoff/sit314-bmoas>. The student arranges tutor access.

Appendix B Throughput and resource use

Final workload	Accepted /s	Completed /s	Empty queues after publication
Baseline	99.99	99.98	31.54 s
Higher load	999.81	998.05	30.79 s
Reconnect	516.58	528.08	28.32 s

Accepted throughput divides MQTT acknowledgements by the simulator interval. The reconnect simulator starts before its first publication, so its denominator is slightly longer than the completion interval. Completed throughput includes the time to the last aggregation confirmation. Queue recovery is the first sample with every observation completed and all processing queues empty. It includes the sampling interval. All three trials had exact identifiers and ratings, with no service error codes recorded in the captured logs.

Stage	Mean CPU	Mean memory	Peak sampled queue
nodered	8.73%	5.49%	72
validator	16.67%	4.88%	100
aggregator	23.80%	7.21%	95

CloudWatch means above use available one minute samples during the final higher-load processing window. Queue peaks are sampled visible plus in-flight messages. Four tasks remained running at each processing stage throughout the final trials. The reporter used one additional task. Preparing capacity allocated more resources than the original one-task starting point.

The reconnect workload included 50,000 buffered observations and 12,000 regular observations. Final commit occurred 0.149 seconds after publication ended, and the next empty-queue sample was at 28.32 seconds. Both measures were within the original 120 second recovery target.

Appendix C Implementation of the further changes

The following code is the complete new reporting method. It belongs to the MongoDB storage component. It groups stored counters by their original ratings bucket, adds observation and participant counts and removes internal partition fields before returning results. The API retains its existing limit of 500 results.

```
async reportRows(filter = {}, limit = 0) {
  const pipeline = [
    { $match: filter },
    {
      $group: {
        _id: { $ifNull: ['$logical_bucket_id', '_id'] },
        record: { $first: '$$ROOT' },
        observation_count: { $sum: '$observation_count' },
        participant_count: { $sum: '$participant_count' },
        updated_at: { $max: '$updated_at' },
      },
    },
    {
      $replaceRoot: {
        newRoot: {
          $mergeObjects: [
            '$record',
            {
              _id: '$_id',
              observation_count: '$observation_count',
              participant_count: '$participant_count',
              updated_at: '$updated_at',
            },
          ],
        },
      },
    },
    { $unset: ['logical_bucket_id', 'shard'] },
    { $sort: { window_start: 1, _id: 1 } },
  ];
  if (limit > 0) pipeline.push({ $limit: limit });
  return this.db.collection('aggregates').aggregate(pipeline).toArray();
}

async reports(filter = {}) {
  return this.reportRows(filter, 500);
}
```

Appendix C Transactional aggregation

This method belongs to the aggregation storage component. It reads the observations, assigns each participant to a stable storage group, adds new memberships and updates counters in one transaction. It then confirms completion. A repeated delivery leaves the counts unchanged. These are the implementation details used in the final cloud image.

```
async aggregateBatchOnce(
  ids,
  { windowSeconds = 60, failBeforeCommit = false, shards = 1 } = {},
) {
  const session = this.client.startSession();
  try {
    const result = await session.withTransaction(
      async () => {
        const observations = this.db.collection('observations');
        const records = await observations
          .find({ _id: { $in: ids } }, { session })
          .toArray();
        if (records.length !== ids.length) {
          throw new Error('observation_not_ready');
        }
        const outcomes = new Map();
        const buckets = new Map();
        const memberships = new Map();
        const updates = [];
        const now = new Date();
        for (const record of records) {
          if (record.completed_at) {
            outcomes.set(record._id, { duplicate: true });
            continue;
          }
          let disposition = 'counted';
          let bucketId = null;
          if (record.late) {
            disposition = 'late_review';
          } else if (record.event.media_source_id === null) {
            disposition = 'no_media';
          } else {
            const bucket = bucketFor(record, windowSeconds);
            bucketId = bucket._id;
            if (
              !Number.isSafeInteger(shards) ||
              shards < 1 ||
              shards > 256
            ) {
              throw new Error('invalid_aggregation_shards');
            }
            let storageId = bucketId;
            if (shards > 1) {
              const shard =
                parseInt(hash(record.event.panel_id).slice(0, 8), 16) %
                shards;
              storageId = hash([bucketId, shard]);
              bucket.logical_bucket_id = bucketId;
              bucket.shard = shard;
              bucket._id = storageId;
            }
            if (!buckets.has(storageId)) {
              buckets.set(storageId, {
                bucket,
                expiresAt: record.expires_at,
                observations: 0,
                participants: 0,
              });
            }
            buckets.get(storageId).observations++;
          }
        }
      }
    );
  }
}
```

```

    const membershipId = hash([
      bucketId,
      record.event.panel_id,
    ]);
    if (!memberships.has(membershipId)) {
      memberships.set(membershipId, {
        bucketId,
        storageId,
        panelId: record.event.panel_id,
        expiresAt: record.expires_at,
      });
    }
  }
  updates.push({
    updateOne: {
      filter: { _id: record._id },
      update: {
        $set: {
          completed_at: now,
          disposition,
          bucket_id: bucketId,
          latency_ms: now - record.published_at,
        },
      },
    },
  });
  outcomes.set(record._id, { duplicate: false, disposition });
}
// One membership write per distinct person and bucket in this
// batch.
// The transaction makes concurrent batches retry before counts
// can diverge.
const memberEntries = [...memberships.entries()];
if (memberEntries.length) {
  const writes = memberEntries.map(([id, member]) => ({
    updateOne: {
      filter: { _id: id },
      update: {
        $setOnInsert: {
          bucket_id: member.bucketId,
          panel_id: member.panelId,
          expires_at: member.expiresAt,
        },
      },
      upsert: true,
    },
  }));
  const inserted = await this.db
    .collection('memberships')
    .bulkWrite(writes, { session });
  for (const index of Object.keys(inserted.upsertedIds)) {
    const member = memberEntries[Number(index)][1];
    buckets.get(member.storageId).participants++;
  }
}
if (buckets.size) {
  const writes = [...buckets.values()].map((group) => ({
    updateOne: {
      filter: { _id: group.bucket._id },
      update: {
        $setOnInsert: {
          ...group.bucket,
          expires_at: group.expiresAt,
        },
        $inc: {
          observation_count: group.observations,
          participant_count: group.participants,
        },
        $set: { updated_at: now },
      },
    },
  }));

```

```

        },
        upsert: true,
    },
    }));
    await this.db
        .collection('aggregates')
        .bulkWrite(writes, { session });
    }
    if (updates.length) {
        await observations.bulkWrite(updates, { session });
    }
    if (failBeforeCommit) {
        throw new Error('injected_transaction_failure');
    }
    return outcomes;
},
{
    readConcern: { level: 'snapshot' },
    writeConcern: { w: 'majority' },
    maxCommitTimeMS: 10000,
},
);
// Confirmation is written only after the whole transaction commits.
// Redelivery repairs an interrupted confirmation without increasing
// ratings.
const confirmed = new Date();
await this.db
    .collection('observations')
    .updateMany(
        { _id: { $in: ids }, confirmed_at: { $exists: false } },
        [
            {
                $set: {
                    confirmed_at: confirmed,
                    processing_latency_ms: {
                        $subtract: [confirmed, '$published_at'],
                    },
                },
            },
        ],
    ),
    ],
);
return result;
} finally {
    await session.endSession();
}
}

```

Appendix D CloudWatch and configuration

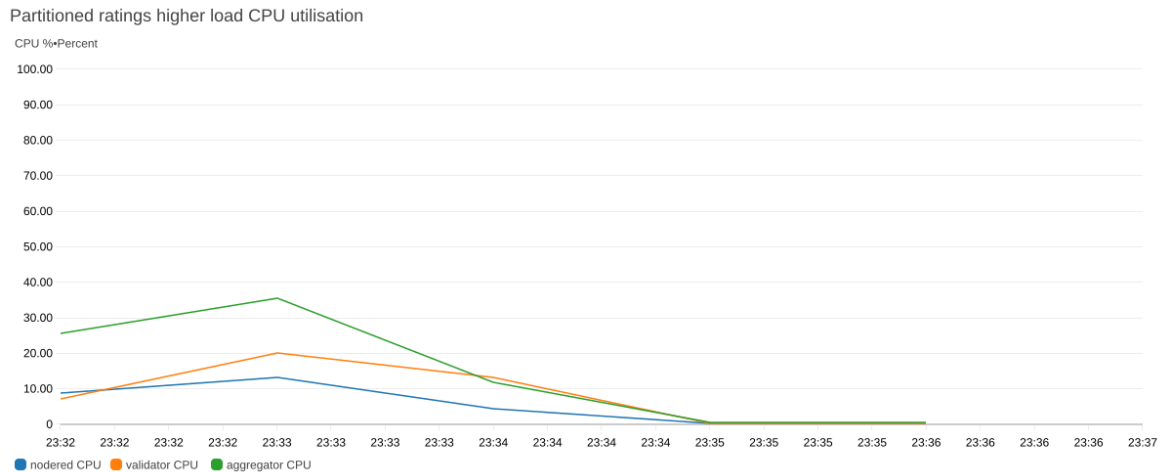


Figure D1. Graph rendered by AWS CloudWatch from the final higher-load trial. Times are UTC. The saved widget configuration identifies the services, interval and metrics.

Capacity configuration

The infrastructure definition provides optional morning, daytime, evening and overnight schedules. Each scheduled action changes the minimum capacity while retaining a maximum of four. Australia/Sydney time accounts for daylight saving. Recurring schedules default to disabled for controlled experiments. The demonstrated one-off action used the same AWS Application Auto Scaling mechanism.

Appendix D Scheduled capacity definition

This excerpt is the scheduled action configuration from the aggregation scaling target. Node-RED and validation use the same actions. The condition enables the timetable only when requested. Schedule parameters and the time zone are supplied by the infrastructure template. The deployment used four tasks per processing stage for the measured trials.

```
{
  "Fn::If": [
    "AudienceSchedule",
    [
      {
        "ScheduledActionName": "audience-morning-news",
        "Schedule": {"Ref": "MorningCapacitySchedule"},
        "Timezone": {"Ref": "CapacityTimezone"},
        "ScalableTargetAction": {
          "MinCapacity": 4,
          "MaxCapacity": 4
        }
      },
      {
        "ScheduledActionName": "audience-daytime",
        "Schedule": {"Ref": "DaytimeCapacitySchedule"},
        "Timezone": {"Ref": "CapacityTimezone"},
        "ScalableTargetAction": {
          "MinCapacity": 2,
          "MaxCapacity": 4
        }
      },
      {
        "ScheduledActionName": "audience-evening-news-sport",
        "Schedule": {"Ref": "EveningCapacitySchedule"},
        "Timezone": {"Ref": "CapacityTimezone"},
        "ScalableTargetAction": {
          "MinCapacity": 4,
          "MaxCapacity": 4
        }
      },
      {
        "ScheduledActionName": "audience-overnight",
        "Schedule": {"Ref": "OvernightCapacitySchedule"},
        "Timezone": {"Ref": "CapacityTimezone"},
        "ScalableTargetAction": {
          "MinCapacity": 1,
          "MaxCapacity": 4
        }
      }
    ],
    {"Ref": "AWS::NoValue"}
  ]
}
```

References

Amazon Web Services (2026), How scheduled scaling for Application Auto Scaling works, AWS documentation, <https://docs.aws.amazon.com/autoscaling/application/userguide/scheduled-scaling-policy-overview.html>.

Amazon Web Services (2026), AWS::ApplicationAutoScaling::ScalableTarget ScheduledAction, AWS CloudFormation documentation, <https://docs.aws.amazon.com/AWSCloudFormation/latest/TemplateReference/aws-properties-applicationautoscaling-scalabletarget-scheduledaction.html>.